Informe Técnico

Desarrollo Backend Avanzado: Spring Cloud y Batch

Modernización del sistema legacy del Banco XYZ a una arquitectura de microservicios en la nube

Asignatura	Desarrollo Backend III (PBY2203) · Evaluación Final Transversal, Semana 9
Autor	Daniel Erices
Repositorio	https://github.com/xXxDaenNoxXx/Backend_III_S7 (rama eft-s9)
Tecnologías	Java 17 · Spring Boot 3.3.5 · Spring Cloud 2023.0.3 · Spring Batch 5.1 · Spring Security / Authorization Server · Resilience4j 2.2 · Apache Kafka · PostgreSQL 16 · Docker Compose · AWS EC2

1. Resumen ejecutivo

El Banco XYZ opera desde hace más de 30 años sobre una plataforma legacy (COBOL y scripts Shell en mainframe) que concentra transacciones, intereses, estados de cuenta y clientes. Sus límites son la escalabilidad, el costo de mantención y la dificultad de integrar tecnologías nuevas. Este proyecto entrega la base de su reemplazo: una plataforma de **16 contenedores** orquestados con Docker Compose y desplegable en AWS.

El alcance cubre las tres partes del caso: **(1)** los tres procesos batch legacy reescritos en Spring Batch; **(2)** el patrón Backend for Frontend para tres canales (web, móvil y cajeros automáticos); y **(3)** tres microservicios de negocio (Clientes, Cuentas y Pagos) con Spring Cloud Config, Eureka, API Gateway, OAuth2, Resilience4j y mensajería asíncrona con Kafka.

Parte del caso	Componente(s) entregado(s)	Estado
1. Batch	batch-service: 3 jobs (transacciones diarias, intereses mensuales, estados anuales)	Implementado
2. BFF	bff-cajeros (:8094), bff-web (:8091), bff-movil (:8092), todos HTTPS + OAuth2	Implementado
3. Microservicios	ms-clientes, ms-cuentas, ms-pagos + config-server, eureka-server, api-gateway, auth-server, Kafka, PostgreSQL	Implementado
Despliegue	docker-compose.yml (16 servicios), CI/CD GitHub Actions → Docker Hub → EC2	Implementado

Nota de verificación. El código fue escrito y revisado de forma estática; en el entorno de desarrollo no fue posible compilar ni ejecutar porque no había acceso a Maven Central. Por eso este informe no incluye métricas de rendimiento medidas. La primera compilación real ocurre en el `docker compose build` y la verificación funcional se hace con `scripts/smoke-test.sh` (sección 10).

2. Procesos clave identificados para la migración

Se identificaron cinco procesos críticos. Cada uno resuelve una limitación concreta del sistema legacy y tiene un componente propio en la solución.

#	Proceso clave	Problema legacy que resuelve	Componente
1	Migración de procesos batch a Spring Batch	Cron + COBOL/Shell sin reintentos, sin trazabilidad de rechazos y de ejecución secuencial	batch-service
2	División del monolito en microservicios	Un fallo en un módulo afecta a todo el sistema; escalar implica escalar todo	ms-clientes, ms-cuentas, ms-pagos, api-gateway, eureka-server, config-server
3	Patrón Backend for Frontend (BFF)	Todos los canales reciben los mismos datos y dependen del tiempo de respuesta del monolito	bff-cajeros, bff-web, bff-movil
4	Seguridad distribuida (Spring Cloud Security / OAuth2)	Seguridad centralizada y limitada: un solo perímetro	auth-server + Resource Server en cada servicio
5	Mensajería asíncrona con Apache Kafka	Acoplamiento síncrono entre módulos; sin eventos en tiempo real	Kafka + productores/consumidores en pagos, cuentas, clientes y BFF cajeros

3. Propuesta de arquitectura

La arquitectura separa responsabilidades en capas: canales → BFF → servicios de negocio → datos y mensajería, con servicios transversales de configuración, descubrimiento, entrada y seguridad.

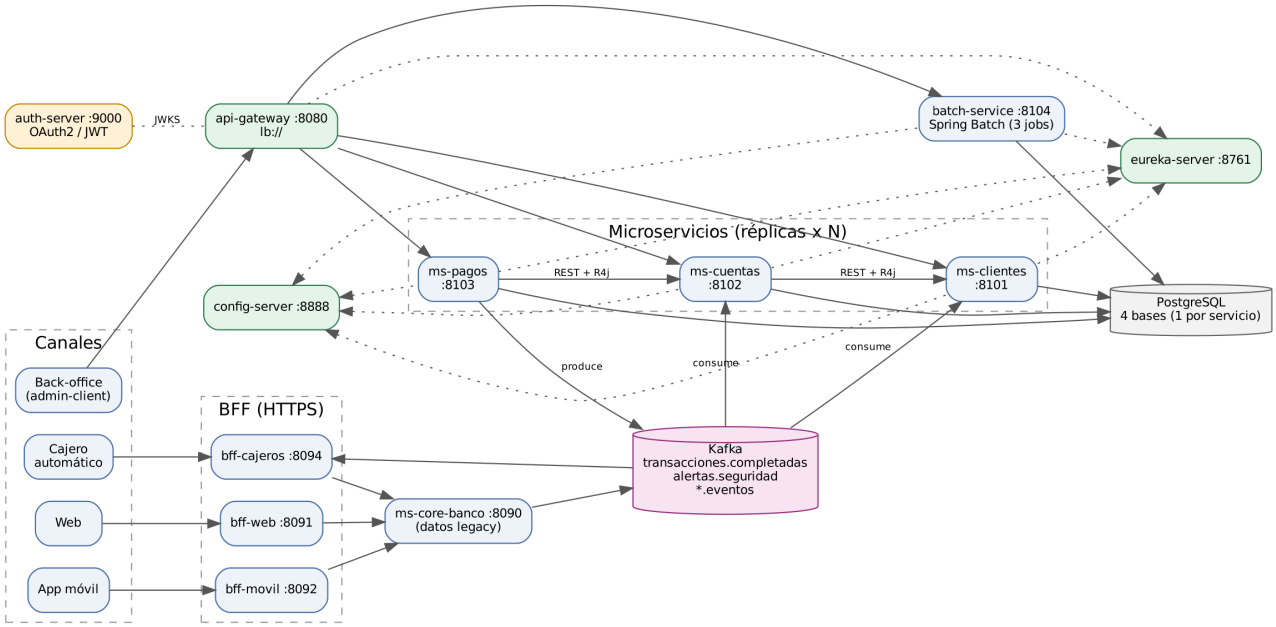


Figura 1. Arquitectura general de la plataforma (líneas punteadas: registro/configuración).

3.1 Requerimientos clave del negocio y decisiones

Requerimiento del negocio	Decisión de arquitectura	Justificación
R1. Escalabilidad y costo. El mainframe escala solo de forma vertical y es caro de mantener.	Microservicios stateless con base propia, detrás de Eureka + Spring Cloud LoadBalancer; escalado con `MS_REPLICAS` o `--scale`. Batch con particionado y steps multi-hilo.	El estado vive en PostgreSQL y Kafka, no en la memoria del servicio: agregar réplicas no requiere cambios de código. Se escala solo el servicio con demanda (p. ej. pagos).
R2. Resiliencia y consistencia. Un módulo caído no debe tumbar el banco y el dinero nunca debe duplicarse ni perderse.	Resilience4j (Retry + CircuitBreaker + fallback) en cada llamada remota; **saga con compensación** en transferencias; operaciones idempotentes por referencia; bloqueo pesimista de la cuenta.	Las transacciones distribuidas (2PC) no escalan ni son soportadas por Kafka/REST. La saga mantiene consistencia eventual con estado persistido y reintentos seguros.
R3. Seguridad por canal y de extremo a extremo. Datos financieros; canales con distinto riesgo.	auth-server OAuth2 (client_credentials, JWT firmado con RSA); autorización por *scope* en cada servicio (no solo “autenticado”); un token de canal no sirve en otro canal; BFF por HTTPS.	Los actores son sistemas (cajeros, apps, servicios), no personas con login; se aplica mínimo privilegio y defensa en profundidad: el gateway no confía, cada servicio valida.
R4. Agilidad de los equipos frontend.	Un BFF independiente por canal, con su propio cliente OAuth2, contrato y despliegue.	Cada canal optimiza su payload (móvil ligero, web completo, cajero mínimo) y se desarrolla en paralelo sin tocar el core.

3.2 Otras decisiones técnicas

Decisión	Motivo
Database-per-service (clientes_db, cuentas_db, pagos_db, batch_db) en una instancia PostgreSQL	Ningún servicio lee tablas de otro: solo se integran por API o eventos. Evita acoplamiento por esquema. En producción cada base puede moverse a su propio RDS.
Config Server en modo *native* (archivos en la imagen)	Centraliza tasas de interés, umbrales y parámetros de Resilience4j. Cambiar a un repositorio Git es solo una propiedad.
API Gateway sin validación de token	Evita un punto único de confianza; reenvía `Authorization` y cada microservicio valida el JWT.
Kafka con clave = id de cuenta	Garantiza orden de los eventos de una misma cuenta y reparto de particiones entre réplicas del mismo grupo.
Spring Boot 3.3.5 en todos los módulos nuevos	Compatible con Spring Cloud 2023.0.x y resilience4j-spring-boot3. El core de la S8 se mantiene sin cambios.

4. Parte 1: migración de procesos batch con Spring Batch

El módulo **batch-service** reescribe los tres procesos legacy usando como entrada los CSV del repositorio `fin_legacy_data` (carpeta `semana_3`, 1.000 filas por archivo con errores intencionales). Cada job es lanzable por cron (reemplaza al cron del mainframe) y por API REST.

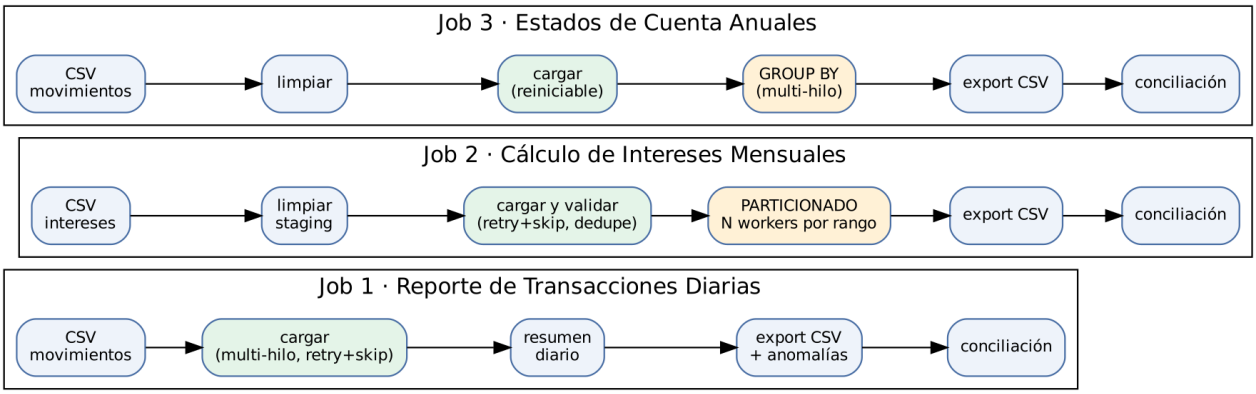


Figura 2. Los tres jobs y sus steps.

Job	Reglas de negocio	Paralelismo / escalabilidad
reporteTransaccionesDiariasJob (legacy: reporte diario)	Clasifica cada transacción como VALIDA o ANOMALA con su motivo (fecha inválida, monto faltante/cero/negativo, tipo distinto de débito/crédito). Las anomalías no se descartan: se guardan y se exportan a CSV. Genera resumen por día (créditos, débitos, balance).	Step multi-hilo (4 hilos) con lector sincronizado y escritura *upsert* idempotente.
calculoInteresesMensualesJob (legacy: cálculo mensual)	Valida cuenta (saldo, edad 18-120, tipo ahorro/préstamo/hipoteca), deduplica por cuenta_id (gana el primer registro válido) y calcula interés = saldo × tasa mensual (tasas en Config Server, redondeo HALF_UP). Ahorro abona; préstamo e hipoteca cargan.	Step **particionado** por rango de cuenta_id: un worker por partición, cada uno con su propio lector.
estadosCuentaAnualesJob (legacy: estados anuales)	Normaliza fechas (4 formatos) y la tilde de “depósito”; rechaza monto faltante o cero. Como el signo del CSV legacy no es confiable, guarda el valor absoluto y el sentido lo da el tipo. Agrega por cuenta y año; saldo neto negativo queda en estado REVISAR.	Carga reiniciable desde el último commit; agregación multi-hilo.

4.1 Manejo de errores, reintentos y reejecución

Mecanismo	Implementación
Reintentos ante fallos temporales	Retry (3 intentos, backoff exponencial 250 ms → 4 s) para TransientDataAccessException, DataAccessResourceFailureException, CannotCreateTransactionException y CannotGetJdbcConnectionException (caída de conexión a la BD).
Registros inválidos	Skip de líneas ilegibles y registros que violan reglas de calidad. Cada rechazo queda en `registro_rechazado` con el job, el step, el registro y el motivo: nada se pierde en silencio.
Umbral de calidad	`batch.skip-limit` (900). Si los rechazos superan el umbral, el step falla: un archivo corrupto no se procesa “a medias” sin que nadie se entere. Con los datos de semana_3 se rechazan ≈578 filas en intereses y ≈164 en estados, bajo el umbral.
Finalización y reejecución automática	JobAutoRestarter: (a) al arrancar marca como FAILED las ejecuciones que quedaron en STARTED por una caída; (b) cada 60 s relanza con los mismos parámetros toda instancia cuya última ejecución falló (restart de Spring Batch: los steps COMPLETED no se repiten y los reiniciables continúan desde el último chunk); (c) tope de 3 reejecuciones, luego alerta en el log para intervención manual.
Integridad y equivalencia con el legacy	Step de **conciliación** (equivalente a los “totales de control” de COBOL): leídos – filtrados – omitidos = escritos, registrado en `conciliacion_batch`. Todos los writers son idempotentes (upsert), por lo que reejecutar no duplica datos.

Limitación declarada: no se dispone de la salida original del sistema legacy; la equivalencia se demuestra con reglas explícitas y conciliación de totales, no con una comparación registro a registro. El archivo de intereses del repositorio se llama “trimestrales”; el job calcula con tasa mensual según el enunciado.

5. Parte 2: patrón Backend for Frontend

Tres BFF independientes (proyecto, imagen y despliegue propios) consumen el mismo core, pero cada uno entrega solo lo que su canal necesita. Todos exponen HTTPS (PKCS12), validan JWT con su propio *scope* y obtienen un token de servicio (client_credentials) para llamar al core.

BFF	Optimización	Seguridad / resiliencia	Endpoints
bff-web :8091	Respuesta completa: titular, historial ordenado y totales ya calculados en una sola llamada (menos viajes de red). Compresión y ETag (304 sin cuerpo).	Scope `web.read`; cliente de servicio con `core.read`	GET /web/cuentas, /{id}, /{id}/resumen
bff-movil :8092	Payload mínimo: saldo + últimos 5 movimientos de 3 campos, sin datos personales. Compresión y `Cache-Control: max-age=15`.	Scope `movil.read`; solo lectura	GET /movil/cuentas/{id}/saldo, /{id}/resumen
bff-cajeros :8094	Operaciones críticas: saldo y retiro. Respuesta mínima. Consume el evento de transacción y genera el comprobante.	Scopes `cajero.read`/`cajero.write`; circuit breaker + retry con fallback a “servicio no disponible”	GET /cajero/cuentas/{id}/saldo, POST /{id}/retiro

Un token de un canal no sirve en otro (403) y sin token todo responde 401; además el core no es accesible con tokens de canal. Los 4xx del core se tratan como errores de negocio y no abren el circuito; las caídas del core responden 503 con `Retry-After`.

6. Parte 3: microservicios resilientes con Spring Cloud

Servicio	Responsabilidad	API principal (scope)	Eventos Kafka
ms-clientes :8101	Información personal y perfiles (BASICO, PREMIUM, EMPRESA). Valida RUT y unicidad.	POST/PUT/PATCH /clientes (clientes.write), GET (clientes.read)	Produce *clientes.eventos*. **Consume** *alertas.seguridad* y las asocia al cliente.
ms-cuentas :8102	Apertura (valida al cliente en ms-clientes), mantenimiento, cierre (solo con saldo 0), débito/crédito idempotente.	POST /cuentas, /{id}/cierre, /debitar, /acreditar (cuentas.write); GET (cuentas.read)	Produce *cuentas.eventos*. **Consume** *transacciones.completadas* (actividad por cuenta).
ms-pagos :8103	Depósitos, pagos y transferencias con saga; alerta de seguridad si el monto supera el umbral.	POST /pagos/deposito, /pago, /transferencia (pagos.write); GET (pagos.read)	Produce *transacciones.completadas* y *alertas.seguridad*.

6.1 Spring Cloud: configuración, descubrimiento y balanceo

Config Server sirve `application.yml` (común) y un archivo por servicio; los clientes lo importan con `spring.config.import` con *fail-fast* y reintentos, de modo que no arrancan sin su configuración. **Eureka** registra cada instancia; los clientes llaman a `http://ms-cuentas/...` con un RestTemplate `@LoadBalanced` (round-robin entre réplicas). El **API Gateway** enruta `/clientes`, `/cuentas`, `/pagos` y `/batch` a `lb://...` y reintenta los GET si una instancia acaba de caer.

6.2 Resiliencia (Resilience4j) y comportamientos alternativos

Llamada	Protección	Comportamiento alternativo (fallback)
ms-cuentas → ms-clientes (validar titular)	Retry 3×, backoff exponencial; CircuitBreaker (ventana 6, 50 %, 10 s abierto)	ServicioNoDisponible → HTTP 503 + Retry-After. Nunca se abre una cuenta sin validar al titular. Cliente inexistente (404) se informa como regla de negocio (422) y no abre el circuito.

Llamada	Protección	Comportamiento alternativo (fallback)
ms-pagos → ms-cuentas (debitar / acreditar)	Igual. El reintento es seguro porque ms-cuentas es idempotente por referencia.	El pago queda RECHAZADO con el motivo, sin mover saldos; el cliente puede reintentar con la misma Idempotency-Key.
BFF → core	Retry + CircuitBreaker	Mensaje de servicio no disponible (503); los 4xx se propagan.

6.3 Consistencia de datos en un entorno distribuido

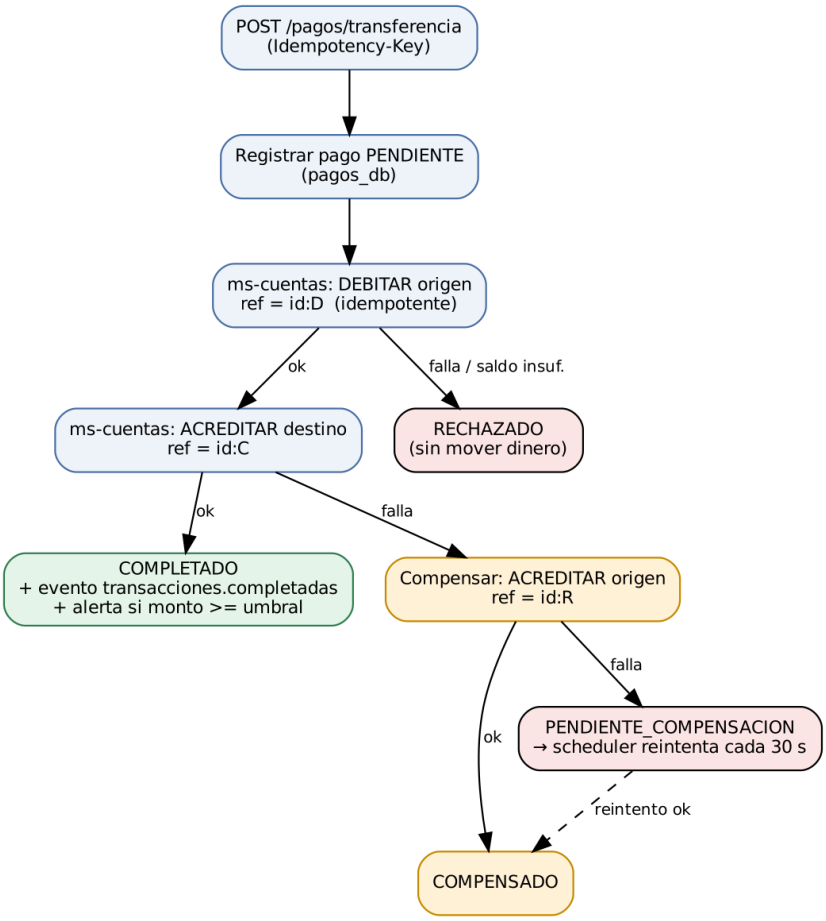


Figura 3. Saga de transferencia con compensación.

Medidas: (1) **idempotencia** en dos niveles: `Idempotency-Key` de la petición (clave primaria del pago) y `referencia` única (cuenta + referencia + tipo) en el libro de movimientos de ms-cuentas; (2) **bloqueo pesimista** (SELECT … FOR UPDATE) de la cuenta, que serializa débitos concurrentes incluso entre réplicas; (3) la **llamada remota se hace fuera de la transacción de base de datos**; (4) el estado de la saga se persiste y un scheduler reintenta las compensaciones pendientes; (5) los eventos Kafka se publican después de confirmar y su falla nunca revierte la operación.

6.4 Monitoreo y registro

Cada servicio expone Actuator (`health`, `metrics`, `circuitbreakers`, `retries`) con sondas usadas por los *healthchecks* de Docker; los eventos de negocio relevantes (evento publicado, alerta registrada, rechazo batch, reintento, conciliación) se registran con prefijos buscables ([BATCH], [RESTART], [SCHEDULER]). Eureka y Kafka UI (:8085) dan visibilidad de instancias y tópicos.

7. Seguridad

Cliente OAuth2	Tipo	Scopes	Uso
cajero-client / web-client / movil-client	Canal	cajero.read+write / web.read / movil.read	Cada canal llama a su BFF
bff-*-service	Servicio	core.read (+ core.write en cajeros)	BFF → ms-core-banco
ms-cuentas-service	Servicio	clientes.read	ms-cuentas → ms-clientes
ms-pagos-service	Servicio	cuentas.read, cuentas.write	ms-pagos → ms-cuentas
admin-client	Back-office	clientes/cuentas/pagos .read/.write, batch.run	Operación vía gateway y lanzamiento de jobs

Flujo **client_credentials**: el token JWT (RSA) lo emite auth-server; cada servicio lo valida contra el JWKS y aplica `hasAuthority("SCOPE_...")` por método HTTP; todo lo no listado es `denyAll`. Los secretos se inyectan por variables de entorno (archivo `.env`); los valores por defecto son solo de demostración. Las comunicaciones de los BFF son HTTPS con certificado PKCS12; dentro de la red de Docker el tráfico va por una red privada y solo se publican los puertos de entrada.

8. Docker, escalabilidad horizontal y despliegue en AWS

Cada módulo tiene un `Dockerfile` multi-etapa (Maven + JDK 17 para compilar; JRE 17 Alpine con usuario sin privilegios para ejecutar). El `docker-compose.yml` define 16 servicios con *healthchecks*, `depends_on` con condiciones, límites de memoria y volúmenes persistentes (PostgreSQL y salida del batch).

Capa	Servicios
Infraestructura	postgres, zookeeper, kafka, kafka-ui
Spring Cloud	config-server, eureka-server, api-gateway
Seguridad	auth-server
Core y canales	ms-core-banco, bff-cajeros, bff-web, bff-movil
Microservicios (escalables)	ms-clientes, ms-cuentas, ms-pagos (réplicas por defecto: 2)
Batch	batch-service (1 instancia; escala con particiones, no con réplicas)

Escalabilidad horizontal. Los tres microservicios no publican puerto fijo ni `container_name`: se registran en Eureka con su IP y el gateway reparte la carga. Se escala con `MS_REPLICAS=3 docker compose up -d` o `docker compose up -d --scale ms-pagos=4`. Los consumidores Kafka de un mismo servicio comparten *group id*, por lo que las particiones se reparten entre réplicas.

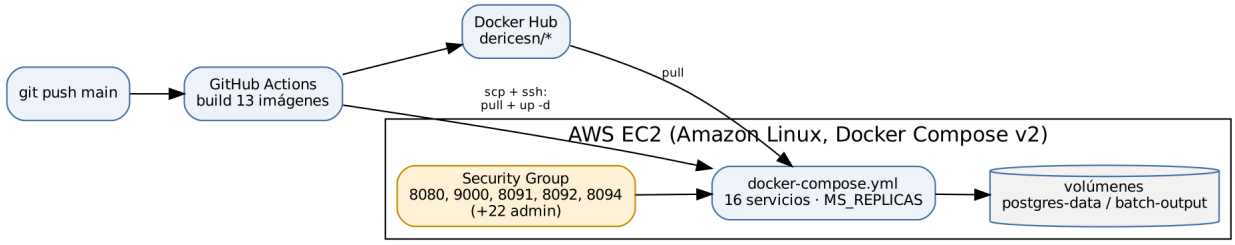


Figura 4. Cadena de despliegue hacia AWS EC2.

8.1 Pasos de despliegue en AWS (resumen)

```
1. EC2: Amazon Linux 2023, minimo t3.large (8 GB RAM) para MS_REPLICAS=2; con 4 GB usar MS_REPLICAS=1.
2. Security Group: abrir 8080 (gateway), 9000 (auth), 8091/8092/8094 (BFF HTTPS), 22 (solo su IP).
3. Instalar Docker y Compose v2 (el binario docker-compose 1.x ignora deploy.replicas).
4. GitHub Secrets: DOCKERHUB_USERNAME, DOCKERHUB_TOKEN, EC2_HOST, EC2_SSH_KEY.
5. git push a main -> Actions construye 13 imagenes, las publica y despliega:
   scp docker-compose.yml  ->  docker compose pull  ->  docker compose up -d --no-build.
6. Opcional: crear ~/.env con los secretos (POSTGRES_PASSWORD, *_SECRET) antes del primer arranque.
```

Para producción real se recomienda sustituir PostgreSQL y Kafka del compose por servicios administrados (RDS, MSK), guardar secretos en AWS Secrets Manager y colocar un Application Load Balancer con certificado ACM delante del gateway.

9. Comparación con el sistema legacy

Aspecto	Legacy (COBOL + Shell, mainframe)	Nuevo sistema
Escalabilidad	Vertical; todo el sistema escala junto	Horizontal por servicio (réplicas) y particionado en batch
Tolerancia a fallos	Un módulo caído afecta a todo; reintento manual	Circuit breaker, retry y fallback por llamada; saga con compensación; reejecución automática de jobs
Procesos batch	Cron + scripts; sin trazabilidad de rechazos	Spring Batch con skip/retry, tabla de rechazos, conciliación y restart desde checkpoint
Canales	Todos reciben el mismo dato del monolito	BFF por canal con payload y seguridad propios
Seguridad	Centralizada y limitada	OAuth2/JWT por scope en cada servicio; HTTPS en los BFF
Integración	Acoplamiento síncrono	Eventos Kafka (desacoplados) + REST con descubrimiento dinámico
Configuración	En scripts/código	Centralizada (Config Server), sin reconstruir imágenes
Despliegue	Manual en mainframe	Contenedores + CI/CD, reproducible

La comparación es cualitativa: no se midieron tiempos de proceso ni del mainframe ni de la nueva plataforma. Una vez desplegada, el campo `leidos/escritos` del endpoint `/batch/jobs/{job}/ultima` y los tiempos de ejecución en el log permiten agregar esas mediciones.

10. Instrucciones de ejecución y prueba

```
git clone https://github.com/xXxDaenNoxXx/Backend_III_S7 && cd Backend_III_S7 && git checkout eft-s9
docker compose up -d --build          # ~5-8 min la primera vez
docker compose ps                     # todos "healthy" (batch y ms-*: ~2 min)
# Eureka: http://<host>:8761   Kafka UI: http://<host>:8085
HOST=localhost ./scripts/smoke-test.sh   # prueba end-to-end con resumen OK/FAIL
```

Pruebas manuales principales (reemplazar TOKEN por el JWT obtenido de `/oauth2/token` con `admin-client:admin123`):

```
# token admin
curl -s -u admin-client:admin123 -d grant_type=client_credentials \
  -d "scope=clientes.read clientes.write cuentas.read cuentas.write pagos.read pagos.write batch.run" \
  http://localhost:9000/oauth2/token

# crear cliente -> abrir cuenta -> transferir (via gateway)
curl -H "Authorization: Bearer TOKEN" -H "Content-Type: application/json" \
  -d '{"rut":"12345678-5","nombre":"Ana","email":"ana@banco.cl"}' http://localhost:8080/clientes
curl -H "Authorization: Bearer TOKEN" -H "Content-Type: application/json" \
  -d '{"clienteId":1,"tipo":"CORRIENTE","saldoInicial":2000000}' http://localhost:8080/cuentas
curl -H "Authorization: Bearer TOKEN" -H "Content-Type: application/json" -H "Idempotency-Key: t-1" \
  -d '{"cuentaOrigenId":1,"cuentaDestinoId":2,"monto":500000}' http://localhost:8080/pagos/transferencia

# batch
curl -X POST -H "Authorization: Bearer TOKEN" http://localhost:8080/batch/jobs/transacciones   # | intereses | estado
curl -H "Authorization: Bearer TOKEN" http://localhost:8080/batch/jobs/transacciones/ultima
curl -H "Authorization: Bearer TOKEN" http://localhost:8080/batch/reportes/conciliacion
docker compose exec batch-service ls /data/output     # reportes CSV generados
```

10.1 Pruebas de resiliencia sugeridas (para el video)

Prueba	Comando	Resultado esperado
Circuit breaker en pagos	docker compose stop ms-cuentas; repetir un pago	503 / pago RECHAZADO; al volver ms-cuentas el circuito se cierra
Escalado horizontal	docker compose up -d --scale ms-pagos=3	Eureka muestra 3 instancias; el gateway reparte las llamadas
Caída de BD en batch	docker compose pause postgres; lanzar job; docker compose unpause postgres	Retry con backoff; si falla, JobAutoRestarter lo reejecuta
Idempotencia	Repetir la misma transferencia con el mismo Idempotency-Key	Misma respuesta; el saldo no cambia dos veces

Prueba	Comando	Resultado esperado
Seguridad	Llamar sin token / con token de otro canal	401 / 403

11. Desafíos enfrentados y soluciones

Desafío	Solución
Datos legacy sucios (4 formatos de fecha, montos vacíos o con signo inconsistente, duplicados, tipos inválidos)	Parseo tolerante (todo texto) + validación en el processor, reglas explícitas, skip con trazabilidad y umbral de calidad.
Evitar dinero duplicado o perdido con servicios independientes y reintentos	Saga con compensación, idempotencia por referencia y Idempotency-Key, bloqueo pesimista y scheduler de compensaciones.
Escalar sin romper la consistencia (H2 en memoria no sirve con réplicas)	PostgreSQL compartido con una base por servicio; servicios sin estado local.
Mezcla de versiones en el repo (core en Spring Boot 4.0.x SNAPSHOT, resto 3.3.5)	Los módulos nuevos usan 3.3.5 + Spring Cloud 2023.0.3 y se integran con el core solo por REST/Kafka; el core no se modificó.
Arranque ordenado de 16 contenedores	Healthchecks y depends_on con condiciones; Config Server con fail-fast y reintentos.
Jobs interrumpidos por caída del contenedor	Recuperación de ejecuciones huérfanas y reejecución automática con tope.
No fue posible compilar en el entorno de desarrollo (sin acceso a Maven Central)	Se reutilizaron las dependencias y patrones ya probados en la S8, se hizo revisión estática y se dejó una prueba de humo automatizada para la primera ejecución.

12. Propuestas de mejora y próximos pasos

Mejora	Beneficio
Patrón Outbox para los eventos Kafka	Publicación garantizada junto con la transacción de base de datos.
Particionado remoto en batch (workers en otros nodos vía Kafka) y lock de líder para el reiniciador	Escalar el procesamiento más allá de un nodo y permitir varias instancias.
Observabilidad: Micrometer + Prometheus + Grafana + trazas distribuidas (OpenTelemetry)	Alertas y diagnóstico de extremo a extremo.
Secrets Manager, RDS, MSK y ALB con HTTPS (ACM) en AWS; orquestación con ECS/EKS	Operación productiva y alta disponibilidad.
Autorización por usuario (Authorization Code + PKCE) en web y móvil	Hoy los canales usan client_credentials; los usuarios finales requerirían identidad propia.
Pruebas automatizadas (JUnit + Testcontainers) y análisis de calidad en el pipeline	Detectar regresiones antes del despliegue.
Migración incremental de los datos del mainframe (Strangler Fig) y Spring Cloud Config con Git + Vault	Reducir riesgo de la transición y gestionar configuración sensible.

Evidencias a adjuntar

Para la entrega se sugiere capturar: `docker compose ps` (todos healthy), panel de Eureka con las réplicas, salida de `smoke-test.sh`, respuesta de `/batch/jobs/*/ultima` y `/batch/reportes/conciliacion`, los CSV generados, el log del circuito abierto y una alerta en Kafka UI. Estas capturas deben tomarse en la EC2 real.